```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sat Nov  2 21:58:28 2024

@author: sebastiencanard
"""

import numpy as np

# La matrice S-box utilisée dans SubBytes
S_BOX = np.array([
    [0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
0x76],
    [0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,
0xc0],
    [0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
0x15],
    [0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
0x75],
    [0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
0x84],
    [0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58,
0xcf],
    [0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
0xa8],
    [0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2],
    [0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73],
    [0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
0xdb],
    [0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
0x79],
    [0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
0x08],
    [0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
0x8a],
    [0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
0x9e],
    [0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
0xdf],
    [0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb,
0x16]
], dtype=np.uint8)

# La matrice inverse Inv S-Box utilisée en déchiffrement
INV_S_BOX = np.array([
    [0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF, 0x40, 0xA3, 0x9E, 0x81, 0xF3, 0xD7,
0xFB],
    [0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E, 0x43, 0x44, 0xC4, 0xDE, 0xE9,
0xCB],
    [0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C, 0x95, 0x0B, 0x42, 0xFA, 0xC3,
0x4E],
    [0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B, 0xA2, 0x49, 0x6D, 0x8B, 0xD1,
0x25],
    [0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D, 0x65, 0xB6,
0x92],
    [0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E, 0x15, 0x46, 0x57, 0xA7, 0x8D, 0x9D,
0x84],
    [0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A, 0xF7, 0xE4, 0x58, 0x05, 0xB8, 0xB3, 0x45,
0x06],
    [0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF, 0xBD, 0x03, 0x01, 0x13, 0x8A,
0x6B],
    [0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA, 0x97, 0xF2, 0xCF, 0xCE, 0xF0, 0xB4, 0xE6,
0x73],
    [0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD, 0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75, 0xDF,
0x6E],
    [0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F, 0xB7, 0x62, 0x0E, 0xAA, 0x18, 0xBE,
```

```
0x1B],
    [0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB, 0xC0, 0xFE, 0x78, 0xCD, 0x5A,
0xF4],
    [0x1F, 0xDD, 0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xEC,
0x5F],
    [0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D, 0xE5, 0x7A, 0x9F, 0x93, 0xC9, 0x9C,
0xEF],
    [0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB, 0xBB, 0x3C, 0x83, 0x53, 0x99,
0x61],
    [0x17, 0x2B, 0x04, 0x7E, 0xBA, 0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0C,
0x7D]
], dtype=np.uint8)

# Transformations SubBytes
def sub_bytes(state):
    return np.vectorize(lambda x: S_BOX[x >> 4][x & 0x0F])(state)

def inv_sub_bytes(state):
    return np.vectorize(lambda x: INV_S_BOX[x >> 4][x & 0x0F])(state)

# Transformations ShiftRows
def shift_rows(state):
    new_state = np.copy(state)
    for i in range(4):
        new_state[i] = np.roll(new_state[i], -i)
    return new_state

def inv_shift_rows(state):
    new_state = np.copy(state)
    for i in range(4):
        new_state[i] = np.roll(new_state[i], i)
    return new_state

# Transformations MixColumns
def mix_columns(state):
    for i in range(4):
        col = state[:, i]
        state[:, i] = [
            gmul(col[0], 2) ^ gmul(col[1], 3) ^ col[2] ^ col[3],
            col[0] ^ gmul(col[1], 2) ^ gmul(col[2], 3) ^ col[3],
            col[0] ^ col[1] ^ gmul(col[2], 2) ^ gmul(col[3], 3),
            gmul(col[0], 3) ^ col[1] ^ col[2] ^ gmul(col[3], 2)
        ]
    return state

def inv_mix_columns(state):
    for i in range(4):
        col = state[:, i]
        state[:, i] = [
            gmul(col[0], 0x0e) ^ gmul(col[1], 0x0b) ^ gmul(col[2], 0x0d) ^ gmul(col[3], 0x09),
            gmul(col[0], 0x09) ^ gmul(col[1], 0x0e) ^ gmul(col[2], 0x0b) ^ gmul(col[3], 0x0d),
            gmul(col[0], 0x0d) ^ gmul(col[1], 0x09) ^ gmul(col[2], 0x0e) ^ gmul(col[3], 0x0b),
            gmul(col[0], 0x0b) ^ gmul(col[1], 0x0d) ^ gmul(col[2], 0x09) ^ gmul(col[3], 0x0e)
        ]
    return state

# Fonction galois pour multiplication dans MixColumns
def gmul(a, b):
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a = (a << 1) & 0xFF
        if hi_bit_set:
            a ^= 0x1B
        b >>= 1
    return p
```

```
# Transformation AddRoundKey
def add_round_key(state, round_key):
    return np.bitwise_xor(state, round_key)


# Calcul de la clé étendue
NB = 4  # Nombre de colonnes dans l'état (4 pour AES)
NK = 4  # Nombre de mots dans la clé d'entrée (4 pour AES-128)
NR = 10 # Nombre de rondes pour AES-128 (10)

# La table des constantes Rcon pour chaque tour
RCON = [
    0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36
]

# Quelques fonctions additionnelles
def sub_word(word):
    """Applique la substitution S-Box sur chaque octet du mot."""
    return [S_BOX[b >> 4][b & 0x0F] for b in word]

def rot_word(word):
    """Rotation circulaire d'un mot de 4 octets (utilisé dans la clé étendue)."""
    return word[1:] + word[:1]

# Calcul effectif de la clé étendue
def expand_key(key, rounds):
    """Génère la clé étendue pour AES."""
    key_symbols = list(key)
    key_schedule = [key_symbols[i * 4:(i + 1) * 4] for i in range(NK)]

    for i in range(NK, NB * (rounds + 1)):
        temp = key_schedule[i - 1]

        if i % NK == 0:
            temp = sub_word(rot_word(temp))
            temp[0] ^= RCON[(i // NK) - 1]

        # Ajout XOR avec le mot NK positions plus tôt
        key_schedule.append([
            key_schedule[i - NK][j] ^ temp[j] for j in range(4)
        ])

    # Transformation en une liste d'octets
    expanded_key = [item for sublist in key_schedule for item in sublist]
    return np.array(expanded_key).reshape(-1, 4, 4)

"""
Question 1.1 - Chiffrement et déchiffrement
"""

# Fonction de chiffrement
def encrypt(plaintext, key, rounds):
    # Initialisation de l'état
    state = np.array(plaintext, dtype=np.uint8).reshape(4, 4)

    """Votre code"""

    return state.flatten().tolist()

# Fonction de déchiffrement
def decrypt(ciphertext, key, rounds):
    """Déchiffrement du texte chiffré AES."""
    # Initialisation de l'état
    state = np.array(ciphertext, dtype=np.uint8).reshape(4, 4)

    """Votre code"""

    return state.flatten().tolist()
```

```
def xor_texts(text1, text2):
    """
    Retourne le XOR de deux textes en clair de même longueur.
    """
    return [b1 ^ b2 for b1, b2 in zip(text1, text2)]

"""
Question 1.2 - Padding et Unpadding
"""

def pkcs7_pad(data, block_size=16):
    """Votre code"""
    return data + padding

def pkcs7_unpad(data, block_size=16):
    """Votre code"""
    return data[:-padding_len]


"""
Question 1.3 - Mode CBC pour un calcul de MAC
"""

def xor_blocks(block1, block2):
    """
    Applique l'opération XOR entre deux blocs de même longueur.
    """
    return [b1 ^ b2 for b1, b2 in zip(block1, block2)]


def cbc_mac(plaintext, key, iv, rounds=10):
    """Votre code"""
    return encrypted_block


"""
Question 1.4 - Mode CTR en chiffrement et déchiffrement
"""

def increment_counter(counter):
    """
    Incrémente un compteur de 16 octets (en supposant un compteur 128 bits).
    """
    counter_int = int.from_bytes(counter, byteorder="big") + 1
    return list(counter_int.to_bytes(16, byteorder="big"))

def ctr_encrypt(data, key, nonce, rounds=10):
    """Votre code"""
    return ctr_crypt(data, key, nonce)

def ctr_decrypt(data, key, nonce, rounds=10):
    """Votre code"""
    return pkcs7_unpad(plaintext)


"""
Question 1.5 - Encrypt-then-MAC
"""

print("--------------------")
print("Question 1.5 - Encrypt-then-MAC avec CTR et CBC")
print("--------------------")

key_mac = [0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0xcf, 0x80, 0x09, 0x89,
0x01, 0x0c]
key_enc = [0x64, 0xa7, 0x08, 0x23, 0xfe, 0xea, 0x86, 0xe1, 0xba, 0x7f, 0x09, 0x56, 0x1a, 0xbb,
0x00, 0xca]
iv = [0x04] * 16  # Exemple d'IV de 16 octets
nonce = [0x03] * 8  # Exemple de nonce de 8 octets
```

```
""" Message initial """
plaintext = [0x32, 0x43, 0xf6, 0xa8, 0x88, 0x5a, 0x30, 0x8d, 0x31, 0x31, 0x98, 0xa2, 0xe0, 0x37,
0x07, 0x34, 0x21, 0x2a, 0x11, 0x9e, 0x1a, 0x00, 0x81, 0x50, 0xb1, 0xe5, 0xee, 0x77, 0x12, 0xe7,
0x70, 0x28]
print("Message avant chiffrement :", plaintext, "\n")

""" Chiffrement du message avec le mode CTR """
ciphertext = ctr_encrypt(plaintext, key_enc, nonce, rounds=10)
print("Texte chiffré en mode CTR :", ciphertext, "\n")

""" Calcul d'un MAC du message avec le mode CBC """
mac = cbc_mac(ciphertext, key_mac, iv, rounds=10)
print("MAC du texte chiffré :", mac, "\n")

""" Déchiffrement du chiffré en mode CTR """
decrypted_text = ctr_decrypt(ciphertext, key_enc, nonce, rounds=10)
print("Texte déchiffré en mode CTR :", decrypted_text, "\n")
```